

HANDS-ON ASSIGNMENT

Module 1: IoT Protocol Integration

SmartFactory Telemetry System

Course Real-Time Data Analytics for IoT	Module Module 2 — Foundations	Total Marks 100 marks
Level Graduate	Estimated Time 8–12 hours	Submission Code + Report

1. Scenario

You have been hired as a platform engineer at SmartFactory Inc., a mid-sized manufacturing company that operates two production lines. The company collects real-time telemetry from sensors on both lines and needs a backend system to ingest, route, and process that data reliably.

Your task is to build the IoT data pipeline from scratch, connecting physical-layer sensors to a backend broker using three different protocols, comparing their behavior, and implementing the backend routing and processing logic.

Factory Setup

- 2 production lines: line1, line2
- 3 sensor types per line: temperature (°C), vibration (mm/s), power consumption (kW)
- 1 alert threshold: temperature > 85°C triggers a CRITICAL alert
- 1 actuator per line: cooling fan (accepts ON/OFF commands)

2. Learning Objectives

By completing this assignment, you will be able to:

- Implement a multi-protocol IoT sensor simulation using MQTT, CoAP, and AMQP.
- Compare QoS / reliability mechanisms across three protocols under identical workloads.
- Design and implement topic-based routing and exchange-based routing for sensor data.
- Analyse real wire-level packet captures and annotate key protocol fields.
- Evaluate protocol trade-offs and write a technically grounded recommendation.

3. Prerequisites & Environment Setup

3.1 Software Requirements

Required Installations

Run the setup script provided in the starter kit: `bash setup.sh`

This installs:

Python 3.10+ · paho-mqtt · aiocoap · pika (AMQP) · influxdb-client · pytest
 Docker & Docker Compose (for Mosquitto, RabbitMQ, InfluxDB, Wireshark-CLI)
 Wireshark / tshark (for packet capture tasks)

3.2 Starting the Infrastructure

```
# Clone the starter kit and start all services
git clone <course-repo>/module1-assignment
cd module1-assignment
docker compose up -d

# Verify services are running
docker compose ps
# Expected: mosquitto (1883/8883), rabbitmq (5672/15672), influxdb (8086)

# Open RabbitMQ management UI
open http://localhost:15672 # user: guest pass: guest
```

4. Assignment Tasks

Task	Title	Marks	Protocol
Task 1	MQTT Sensor Publisher & Subscriber	20	MQTT
Task 2	CoAP Sensor Resource & Observer	20	CoAP
Task 3	AMQP Exchange Routing & Dead Letters	20	AMQP
Task 4	Packet Capture & Protocol Analysis	20	All
Task 5	Protocol Comparison Report	20	All
	TOTAL	100	

Task 1: MQTT Sensor Publisher & Subscriber [20 marks]

Objective

Implement a multi-sensor MQTT publisher and a wildcard subscriber. Demonstrate all three QoS levels and analyze their behavior under simulated packet loss.

1.1 Sensor Publisher (8 marks)

Complete the skeleton file `src/mqtt/publisher.py`. Your publisher must:

- Connect to the Mosquitto broker at `localhost:1883` with a persistent session (Clean Session = 0).
- Publish simulated readings for all 6 sensors (3 types × 2 lines) at 1-second intervals.
- Use the correct topic hierarchy: `factory/{line}/{sensor_type}` (e.g. `factory/line1/temperature`).
- Publish temperature readings at QoS 1, vibration at QoS 0, and power at QoS 2.
- Set a Last Will and Testament: topic = `factory/{line}/status`, payload = 'offline', QoS = 1, retain = True.
- On startup, publish a retained 'online' message to `factory/{line}/status`.
- Log each published message with timestamp, topic, value, and QoS level.

1.2 Wildcard Subscriber (7 marks)

Complete `src/mqtt/subscriber.py`. Your subscriber must:

- Subscribe to `factory/#` to receive all sensor readings and status messages.
- Separately subscribe to `factory/+/temperature` with QoS 2.
- Print each received message showing topic, payload (parsed as JSON), QoS level, and retain flag.
- Detect temperature readings > 85°C and print a formatted CRITICAL ALERT message.
- Track message counts per topic and print a summary every 30 seconds.

1.3 QoS Comparison Experiment (5 marks)

Run the test harness provided in `tests/mqtt/test_qos_loss.py`. This test artificially drops packets using Linux traffic control (tc). You must:

- Run the test with 10% packet loss on the loopback interface.
- Record: messages sent, received at QoS 0, QoS 1, QoS 2 over a 60-second window.
- Complete the results table in your report (Section 5.1).
- Answer the analysis questions provided in the starter kit.

Starter File

src/mqtt/publisher.py — skeleton with TODO comments and type hints

src/mqtt/subscriber.py — skeleton with TODO comments

tests/mqtt/test_qos_loss.py — pre-written test harness (do not modify)

Task 2: CoAP Sensor Resource & Observer [20 marks]

Objective

Implement a CoAP server exposing sensor resources and an observer client that subscribes to live data. Implement Block-wise transfer for a firmware manifest resource.

2.1 CoAP Resource Server (9 marks)

Complete `src/coap/server.py`. Your server must expose the following resources:

URI Path	Methods	Content-Format	Notes
<code>/factory/line1/temperature</code>	GET (Observable)	50 (JSON)	Returns current temp
<code>/factory/line1/vibration</code>	GET (Observable)	50 (JSON)	Returns current vib
<code>/factory/line1/power</code>	GET	50 (JSON)	Returns current kW
<code>/factory/line2/temperature</code>	GET (Observable)	50 (JSON)	Returns current temp
<code>/actuator/line1/fan</code>	GET, PUT	50 (JSON)	ON/OFF state control
<code>/factory/manifest</code>	GET	50 (JSON)	Large Block2 resource

- Sensor values must update every 5 seconds using `asyncio`; Observe subscribers receive every update.
- The `/actuator/line1/fan` resource must accept PUT with payload `{"state": "ON"}` or `{"state": "OFF"}` and return 2.04 Changed.
- The `/factory/manifest` resource must return a JSON document ≥ 3 KB to trigger Block2 transfer.

2.2 Observer Client (7 marks)

Complete `src/coap/observer.py`. Your observer client must:

- Register Observe subscriptions on both temperature resources simultaneously.
- Print each notification showing resource URI, Observe sequence number, value, and arrival timestamp.
- Detect and log stale notifications (sequence number lower than the last received).
- After 60 seconds, deregister both subscriptions cleanly using `Observe = 1`.
- Perform a GET request to `/factory/manifest` and reassemble the Block2 response, printing total bytes received and number of blocks.

2.3 CoAP vs HTTP Proxy (4 marks)

Run the provided integration test in `tests/coap/test_proxy.py` which starts a CoAP-HTTP proxy (using aiocoap's built-in proxy support). You must:

- Verify that a standard HTTP GET to `http://localhost:8080/factory/line1/temperature` returns the same JSON as a direct CoAP GET.
- Document the HTTP response headers and explain which CoAP options they map from (in your report, Section 5.2).

Starter File

`src/coap/server.py` — skeleton with resource class stubs

`src/coap/observer.py` — skeleton with TODO comments

`tests/coap/test_proxy.py` — pre-written proxy integration test

[IGNORE AMQP]**Task 3: AMQP Exchange Routing & Dead Letters [20 marks]****Objective**

Configure a RabbitMQ topology for the SmartFactory backend. Implement topic exchange routing, a Dead Letter Exchange, and a Publisher Confirm producer.

3.1 Broker Topology Setup (5 marks)

Complete `src/amqp/topology.py`. Declare the following topology using the pika library:

Name	Type	Durable	Binding / Notes
<code>iot.telemetry</code>	topic exchange	Yes	Main exchange for all sensor data
<code>iot.dlx</code>	direct exchange	Yes	Dead letter destination
<code>alerts-queue</code>	queue	Yes	Bound to <code>iot.telemetry</code> with key <code>#.critical</code>
<code>temperature-queue</code>	queue	Yes	Bound with key <code>*.*.temperature</code> ; TTL 60s
<code>all-telemetry-queue</code>	queue	Yes	Bound with key <code>factory.#</code> ; max-length 10000
<code>dead-letter-queue</code>	queue	Yes	Bound to <code>iot.dlx</code> ; inspect failed messages
<code>line1-queue</code>	queue	Yes	Bound with key <code>factory.line1.#</code>

Configure `temperature-queue` with `x-dead-letter-exchange = 'iot.dlx'` and `x-dead-letter-routing-key = 'dead'`. Configure `all-telemetry-queue` with `x-max-length = 10000` and `x-overflow = 'dead-letter'`.

3.2 Publisher with Confirms (8 marks)

Complete `src/amqp/producer.py`. Your producer must:

- Connect to RabbitMQ over TLS (port 5671) using the test certificates in `certs/`.
- Enable Publisher Confirms on the channel.
- Publish sensor readings with the correct routing key: `factory.{line}.{sensor_type}` (e.g. `factory.line1.temperature`).
- Set `delivery_mode = 2` (persistent) for temperature and power readings; `delivery_mode = 1` (transient) for vibration.
- Use the message expiration property to set a 60-second TTL on all messages.
- Include a 'critical' tag in the routing key when temperature > 85°C: `factory.line1.temperature.critical`.

- Implement async confirm callbacks and log the delivery tag when a PUBACK (confirm) is received.
- Track and report: total published, confirmed, and unconfirmed counts.

3.3 Consumer with ACK and DLX Inspection (7 marks)

Complete `src/amqp/consumer.py`. Your consumer must:

- Consume from `all-telemetry-queue` with `prefetch_count = 5`.
- ACK each successfully processed message.
- Simulate a 10% random processing failure rate: NACK with `requeue = False` on failure (routes to DLX).
- On receiving a critical alert (routing key ends in `.critical`), print an alert, then ACK.
- Every 30 seconds, poll the dead-letter-queue and print all dead-lettered messages with their `x-death` header metadata.

Starter Files

`src/amqp/topology.py` — skeleton with declaration stubs
`src/amqp/producer.py` — skeleton with confirm callback stubs
`src/amqp/consumer.py` — skeleton with ACK/NACK stubs

Task 4: Packet Capture & Protocol Analysis [20 marks]

Objective

Capture live traffic from all three protocols, annotate packet fields, and answer wire-level analysis questions.

4.1 Capture Setup (2 marks)

Use tshark (command-line Wireshark) to capture traffic. Run the provided capture script:

```
# Capture MQTT, CoAP, and AMQP traffic simultaneously (30 seconds)
bash scripts/capture.sh

# This produces three pcap files:
#   captures/mqtt.pcap   (port 1883)
#   captures/coap.pcap   (port 5683)
#   captures/amqp.pcap   (port 5672) [Ignore]

# Open a capture for analysis
tshark -r captures/mqtt.pcap -V | head -100
```

4.2 MQTT Packet Annotation (6 marks)

From your mqtt.pcap, locate:

1. A CONNECT packet. Fill in the annotation table in report/packet_analysis.md: Frame type byte, remaining length bytes, protocol name bytes, protocol version byte, connect flags byte (expand all 8 bits), keep-alive value, and Client ID.
2. A QoS 1 PUBLISH packet. Annotate: the fixed header byte (expand all 8 bits), remaining length, topic length field, topic string bytes, Packet Identifier, and payload.
3. The corresponding PUBACK. Confirm the Packet Identifier matches.

4.3 CoAP Packet Annotation (6 marks)

From your coap.pcap, locate:

4. A CON GET request. Annotate: the 4-byte fixed header (expand all bits of byte 0), Token bytes, and any Uri-Path option fields (show delta encoding).
5. The corresponding ACK 2.05 Content response. Annotate: fixed header, Token (confirm it matches), Content-Format option, Payload Marker byte (0xFF), and payload.
6. An Observe notification. Identify the Observe option number and sequence value.

4.4 AMQP Frame Annotation (6 marks) [Ignore]

From your amqp.pcap, locate:

7. A basic.publish Method frame. Annotate: Frame Type byte, Channel bytes, Payload Size bytes, Class ID, Method ID, exchange name, routing key, mandatory flag, and Frame End byte.
8. The corresponding Content Header frame. Annotate: Class ID, body_size, property flags, delivery_mode, and content_type.
9. A Heartbeat frame. Note its size and identify the channel number. Explain why the payload is empty.

Task 5: Protocol Comparison Report [20 marks]

Objective

Write a 1,500–2,000 word technical report comparing the three protocols based on your implementation experience and measured results.

Submit your report as report/comparison_report.md (Markdown). The report must include the following sections:

5.1 QoS Comparison Results Table (4 marks)

Complete the following table with measured values from your Task 1.3 QoS experiment (60 seconds, 10% packet loss):

Protocol / QoS	Sent	Received	Lost (%)	Duplicates	Latency (ms)
MQTT QoS 0	—	—	—	—	—
MQTT QoS 1	—	—	—	—	—
MQTT QoS 2	—	—	—	—	—
CoAP NON	—	—	—	—	—
CoAP CON	—	—	—	—	—
AMQP (auto-ack-off)	==	==	==	==	==

5.2 CoAP-HTTP Proxy Mapping (3 marks)

Document the HTTP headers returned by the CoAP-HTTP proxy and map each to its CoAP option equivalent. Complete the table:

HTTP Header	CoAP Option	Your Observed Value
Content-Type	—	—
Cache-Control: max-age	—	—
ETag	—	—
Location	—	—

5.3 Protocol Selection Recommendation (8 marks)

Write a recommendation (500–700 words) advising the SmartFactory CTO on which protocol to use for each data path below. You must justify each recommendation with specific technical evidence from your implementation and packet captures.

Data Path	Your Recommended Protocol + Justification
Sensor → Cloud (high frequency, <100 ms latency)	
Actuator commands (safety-critical, exactly-once)	
Backend service-to-service routing	
OTA firmware delivery to constrained MCU (Class 2)	

5.4 Reflection (5 marks)

In 300–400 words, reflect on:

- One technical challenge you encountered in the implementation and how you resolved it.
- The most surprising difference you observed between the protocols during the packet capture task.
- Which protocol you found the most complex to implement correctly, and what specifically made it harder.

6. Submission Guidelines

6.1 Repository Structure

Submit your work as a ZIP of the complete project directory. Your submission must match this structure:

```

module1-assignment/
├── src/
│   ├── mqtt/
│   │   ├── publisher.py      ← Task 1.1 (completed)
│   │   └── subscriber.py    ← Task 1.2 (completed)
│   ├── coap/
│   │   ├── server.py        ← Task 2.1 (completed)
│   │   └── observer.py      ← Task 2.2 (completed)
│   └── amqp/
│       ├── topology.py      ← Task 3.1 (completed)
│       ├── producer.py      ← Task 3.2 (completed)
│       └── consumer.py       ← Task 3.3 (completed)
├── captures/
│   ├── mqtt.pcap            ← Task 4 capture
│   ├── coap.pcap
│   └── amqp.pcap
├── report/
│   ├── packet_analysis.md   ← Task 4 annotations
│   └── comparison_report.md ← Task 5 report
├── tests/                   ← Do not modify
├── docker-compose.yml       ← Do not modify
└── README.md                ← Brief run instructions

```

6.2 Automated Tests

Before submitting, run the full test suite to verify your implementation:

```

# Run all tests
pytest tests/ -v --tb=short

# Expected output:
# tests/mqtt/test_publisher.py::test_connect_persistent      PASSED
# tests/mqtt/test_publisher.py::test_publishes_correct_topics PASSED
# tests/mqtt/test_publisher.py::test_lwt_configured          PASSED
# tests/mqtt/test_subscriber.py::test_wildcard_subscription   PASSED
# tests/mqtt/test_subscriber.py::test_critical_alert_fires    PASSED
# tests/coap/test_server.py::test_get_temperature             PASSED
# tests/coap/test_server.py::test_put_actuator                PASSED
# tests/coap/test_server.py::test_block2_manifest             PASSED
# tests/amqp/test_topology.py::test_exchanges_declared    PASSED

```

```
# tests/amqp/test_topology.py::test_queues_and_bindings PASSED
# tests/amqp/test_topology.py::test_dlx_routing PASSED
```

Important

All tests must pass before submission. Partial passes reduce marks proportionally as detailed in the marking rubric below.

7. Marking Rubric

Task 1: MQTT [20 marks]

Criterion	Marks	Your Score
Publisher connects with persistent session and correct Client ID	1	
Correct topic hierarchy for all 6 sensors	2	
Correct QoS per sensor type (temp=1, vib=0, power=2)	2	
LWT configured with correct topic, payload, QoS, and retain	1	
Retained 'online' message published on startup	1	
Logging includes all required fields	1	
Subscriber wildcard subscription receives all topics	2	
Subscriber separate QoS 2 temperature subscription works	2	
Critical alert fires at > 85°C with correct formatting	2	
QoS experiment: results table complete and accurate	3	
QoS experiment: analysis answers are correct and well-reasoned	3	

Task 2: CoAP [20 marks]

Criterion	Marks	Your Score
All 6 resources declared and respond to GET	2	
Sensor values update every 5 s; observable resources push updates	2	
PUT on /actuator/line1/fan accepts ON/OFF and returns 2.04	2	
/factory/manifest returns >= 3 KB triggering Block2 transfer	3	
Observer subscribes to both temp resources simultaneously	2	

Observer logs sequence numbers and detects stale notifications	2	
Observer deregisters cleanly after 60 s	1	
Block2 reassembly: correct byte count and block count logged	2	
CoAP-HTTP proxy test passes; HTTP header mapping correct in report	4	

Task 3: AMQP [20 marks]

Criterion	Marks	Your Score
All exchanges declared with correct type and durability	2	
All queues declared with correct arguments (TTL, max-length, DLX)	2	
Bindings correct (routing keys match specification)	1	
Producer connects with TLS and SASL EXTERNAL	1	
Publisher confirms enabled; confirm callback logs delivery tags	2	
Correct delivery_mode per sensor type; correct routing keys	2	
Critical routing key suffix applied when temp > 85°C	1	
Consumer prefetch = 5; manual ACK on all messages	2	
10% random NACK with requeue=False routes to DLX correctly	3	
DLX poll every 30 s; x-death metadata printed	2	
All topology tests pass	2	

Tasks 4 & 5: Analysis & Report [40 marks]

Criterion	Marks	Your Score
Packet captures produced for all three protocols	2	

MQTT CONNECT annotated correctly (all 8 fields)	3	
MQTT PUBLISH (QoS 1) and PUBACK annotated; Packet IDs match	3	
CoAP GET request annotated (header byte expansion correct)	3	
CoAP ACK 2.05 annotated; Token match confirmed; payload identified	3	
AMQP basic.publish Method frame annotated (all fields)	3	
AMQP Header + Body frames annotated; body_size cross-checked	3	
QoS results table complete with accurate measured values	4	
CoAP-HTTP header mapping table correct	3	
Protocol recommendations: each justified with specific technical evidence	8	
Reflection: addresses all three points, technically specific	5	

Academic Integrity

All submitted code must be your own work. You may use the course materials, official library documentation, and RFC specifications as references. You may discuss high-level concepts with classmates in your group. Use of AI-generated code submissions without disclosure is a violation of academic integrity policy.

Questions about the assignment should be posted to the course forum.